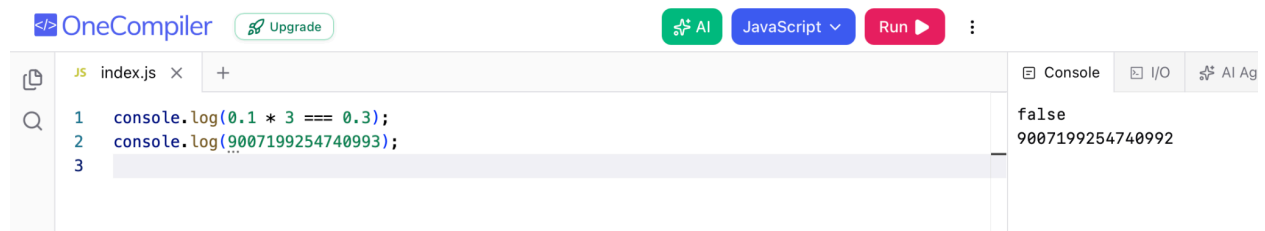


1- JavaScript

(a)



The screenshot shows the OneCompiler JavaScript editor. The code in index.js is:

```
1 console.log(0.1 * 3 === 0.3);
2 console.log(9007199254740993);
3
```

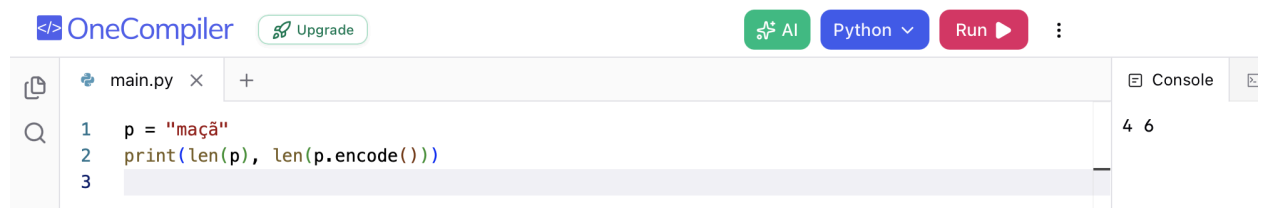
The console output on the right shows:

```
false
9007199254740992
```

(b) Problema: O JavaScript possui problemas de precisão com números decimais. Além disso, 9007199254740993 é maior do que o maior número inteiro que o JavaScript consegue representar exatamente.

2- Python

(a)



The screenshot shows the OneCompiler Python editor. The code in main.py is:

```
1 p = "maçã"
2 print(len(p), len(p.encode()))
3
```

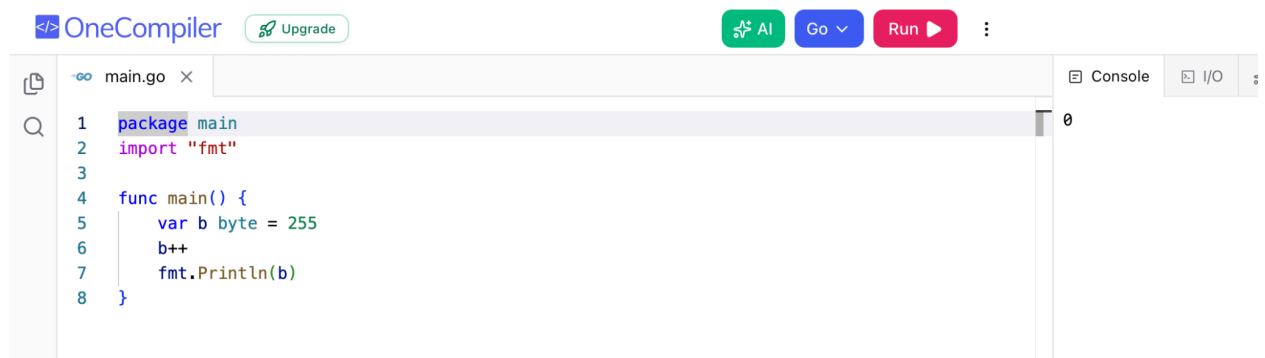
The console output on the right shows:

```
4 6
```

(b) Problema: Não há problema na execução. len(m) conta os caracteres e dá 4. Já encode() transforma o texto em bytes, e por isso o resultado é 5.

3- Go

(a)



The screenshot shows the OneCompiler Go editor. The code in main.go is:

```
1 package main
2 import "fmt"
3
4 func main() {
5     var b byte = 255
6     b++
7     fmt.Println(b)
8 }
```

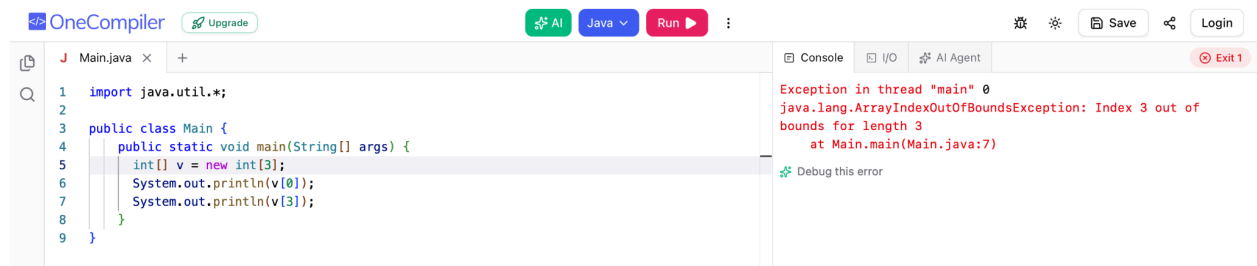
The console output on the right shows:

```
0
```

(b) Problema: Não há problema. O tipo byte pode armazenar valores de 0 até 255.

4- Java

(a)



```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         int[] v = new int[3];
6         System.out.println(v[0]);
7         System.out.println(v[3]);
8     }
9 }
```

Exception in thread "main" 0
java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3
at Main.main(Main.java:7)
Debug this error

(b) Problema: O vetor possui 3 posições. Os índices válidos são 0, 1 e 2. O índice 3 não existe.

5- Rust

(a)



```
1 fn main() {
2     let s = String::from("oi");
3     let t = s;
4     println!("{}", s, t);
5 }
```

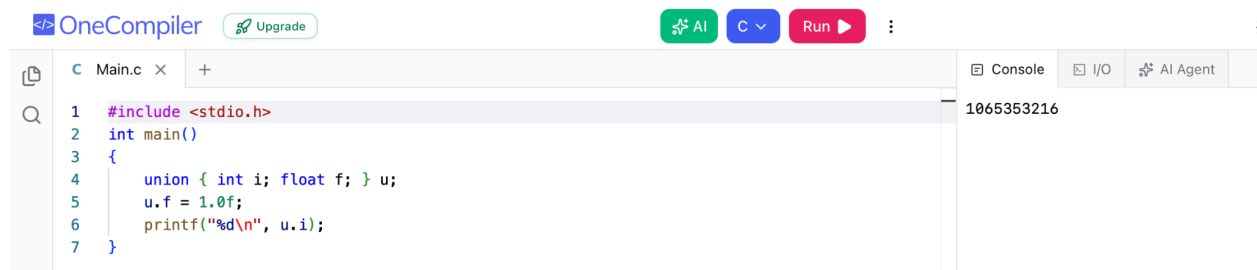
error[E0382]: borrow of moved value: `s`
--> HelloWorld.rs:4:23
|
2 | let s = String::from("oi");
| - move occurs because `s` has type `String`, which
| does not implement the `Copy` trait
3 | let t = s;
| - value moved here
4 | println!("{}", s, t);
| ^ value borrowed here after move
|
help: consider cloning the value if the performance cost is acceptable
|
3 | let t = s.clone();
| +++++++
|
error: aborting due to 1 previous error

For more information about this error, try `rustc --explain E0382`.

(b) Problema: Em Rust, quando fazemos "let t = s", o valor de s é movido para t. Depois disso, não podemos mais usar s. O compilador apresenta um erro de "moved value".

6- C

(a)



```
1 #include <stdio.h>
2 int main()
3 {
4     union { int i; float f; } u;
5     u.f = 1.0f;
6     printf("%d\n", u.i);
7 }
```

1065353216

(b) Problema: i e f utilizam o mesmo espaço de memória porque são uma union. O valor 1.0 foi armazenado como float e depois foi lido como int. Por isso aparece um número diferente.